

REPORTE DE PRÁCTICA

EcoPoints: Sistema de Reciclaje NFC + Botón

Tecnologías Inalámbricas

Fecha: 11/05/2026

Integrantes:

Eduardo Cadengo López
Itzel Citlalli Martell De La Cruz
Damian Alexander Diaz Piña

Contenido

1. Datos Generales	3
2. Descripción de la Tecnología RFID/NFC	3
3. Usos Actuales de la Tecnología	3
4. Funcionamiento y Descripción del Módulo PN532	4
Principio de operación	4
Características principales.....	4
5. Diagrama de Conexión (ESP32 – PN532 – Botón).....	4
Tabla de pines	4
6. Código Arduino IDE — Firmware ESP32 (esp32_nfc_boton.ino)	5
Protocolo serial hacia Python	6
Funciones principales del firmware.....	6
Código completo (Arduino IDE)	6
7. Código Python — Aplicación de Escritorio (app_reciclaje_nfc_v2.py)	8
Dependencias.....	8
Arquitectura de pantallas	8
Flujo completo del sistema	9
Aspectos clave del código Python	9
8. Base de Datos — nfc_reciclaje (MySQL)	10
Tablas del sistema.....	10
Vistas y procedimientos almacenados.....	10
Obtener el UID de una tarjeta nueva	11
9. Conclusiones.....	11
10. Videos, GitHub y Evidencias	12

1. Datos Generales

Práctica	EcoPoints — Recicladora con login NFC y registro de botellas por botón físico
Plataforma	ESP32
Módulo NFC	PN532 (comunicación I2C: SDA=GPIO21, SCL=GPIO22)
Botón físico	2 pines, GPIO 25 con INPUT_PULLUP interno (no requiere resistencia externa)
Entorno Arduino	Arduino IDE + librería Adafruit_PN532
Lenguaje externo	Python 3 — Tkinter + pyserial + mysql-connector-python
Base de datos	MySQL (Laragon) — base de datos: nfc_reciclaje
Objetivo	Implementar una recicladora inteligente: el usuario se identifica acercando su tarjeta NFC y acumula puntos cada vez que presiona el botón físico (simulando el depósito de una botella). Las tarjetas nuevas se registran automáticamente en la base de datos.

2. Descripción de la Tecnología RFID/NFC

RFID (Radio Frequency Identification) es una tecnología de identificación inalámbrica que utiliza radiofrecuencia para leer o escribir información en una etiqueta (tag) mediante un lector. En alta frecuencia (HF) opera a 13.56 MHz y se usa ampliamente en sistemas de identificación y control de acceso.

NFC (Near Field Communication) es una variante especializada de RFID HF orientada a distancias muy cortas (menores a 10 cm). Opera sobre el estándar ISO/IEC 14443, donde el lector genera el campo electromagnético y la tarjeta pasiva responde con su identificador único (UID).

En EcoPoints, el UID de la tarjeta actúa como credencial de identidad del usuario: al acercar la tarjeta al lector PN532, el sistema busca ese UID en la base de datos. Si existe, inicia sesión directamente; si no existe, abre un formulario de registro para crear la cuenta y vincular la tarjeta al nuevo usuario.

3. Usos Actuales de la Tecnología

- Control de acceso a edificios, laboratorios u oficinas mediante credenciales personales.
- Pagos sin contacto y validación rápida en puntos de venta (tarjetas bancarias NFC).

- Tarjetas de transporte público: validación de entrada y salida en autobuses y metros.
- Identificación, trazabilidad e inventarios en bibliotecas, almacenes y logística.
- Programas de reciclaje y gamificación ambiental: acumulación de puntos por depósito de residuos (caso EcoPoints).
- Autenticación en dispositivos móviles, llaves de hotel y accesos industriales.

4. Funcionamiento y Descripción del Módulo PN532

El PN532 es un controlador NFC integrado de NXP Semiconductors que opera a 13.56 MHz. Soporta tarjetas ISO/IEC 14443A (MIFARE Classic, Ultralight, NTAG) y se comunica con el microcontrolador mediante I2C, SPI o UART. En EcoPoints se emplea el modo I2C (SDA/SCL).

Principio de operación

- **1:** Campo RF: el módulo genera un campo electromagnético a 13.56 MHz.
- **2:** Energización: cuando una tarjeta pasiva entra al campo, se energiza por inducción electromagnética.
- **3:** Detección/Selección: el PN532 ejecuta el protocolo ISO14443A (REQA → ATQA → anticollision → SELECT).
- **4:** Entrega del UID: el número único de la tarjeta se transfiere al ESP32, que lo envía por serial ("UID CONCATENADO:XXXXXXXX").

Características principales

- Voltaje de operación: 3.3 V (algunos módulos incluyen regulador a 5 V).
- Frecuencia: 13.56 MHz (HF). Interfaces disponibles: I2C (esta práctica), SPI y UART/HSU.
- Soporta tarjetas ISO14443A/B y FeliCa; también modo Peer-to-Peer (NFC Forum).
- Librería Arduino: Adafruit_PN532 (disponible en el Gestor de Librerías del IDE).
- Cooldown de lectura implementado en firmware: ignora el mismo UID durante 2500 ms para evitar registros duplicados.

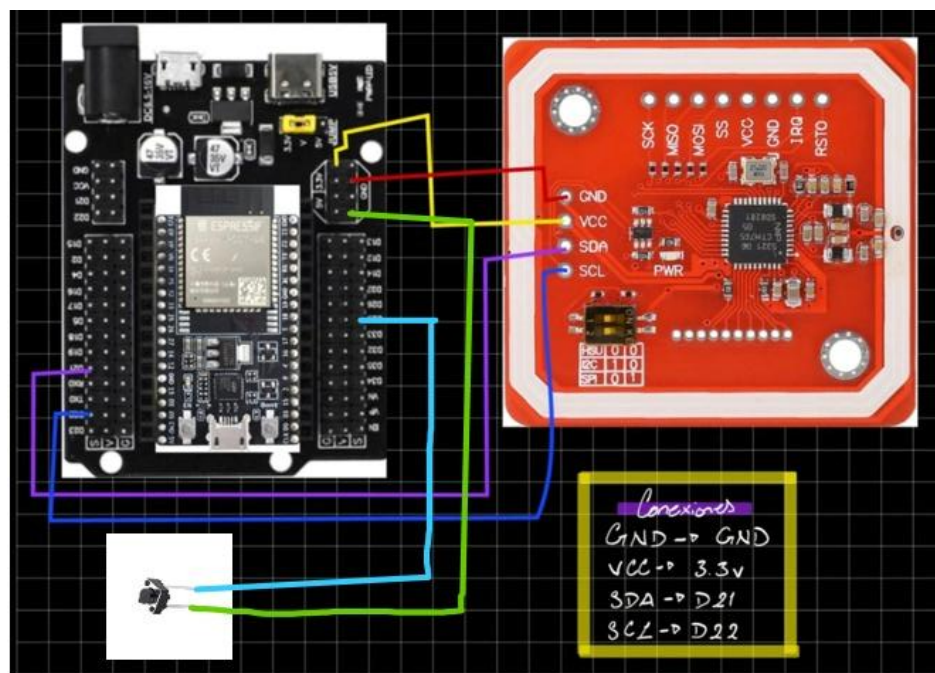
5. Diagrama de Conexión (ESP32 – PN532 – Botón)

Tabla de pines

La siguiente tabla muestra todas las conexiones necesarias entre el ESP32, el módulo PN532 y el botón físico de dos pines:

Componente	Pin del componente	GPIO del ESP32
PN532	VCC	3.3V
PN532	GND	GND
PN532	SDA	GPIO 21
PN532	SCL	GPIO 22
Botón (2 pines)	Pin A	GPIO 25 (INPUT_PULLUP)
Botón (2 pines)	Pin B	GND

Nota importante: el botón no requiere resistencia pull-up externa porque el ESP32 activa su resistencia interna de pull-up mediante INPUT_PULLUP. Esto significa que el pin lee HIGH en reposo y LOW cuando el botón es presionado (lógica invertida). El firmware incluye anti rebote (debounce) de 300 ms para evitar registros múltiples por una sola pulsación.



6. Código Arduino IDE — Firmware ESP32 (esp32_nfc_boton.ino)

El firmware cumple dos funciones simultáneas dentro del loop() principal: leer tarjetas NFC con el módulo PN532 y detectar pulsaciones del botón físico. Ambas funciones

corren de forma no bloqueante gracias al parámetro timeout de 100 ms en readPassiveTargetID().

Protocolo serial hacia Python

El ESP32 comunica dos tipos de eventos al programa Python a 115200 baudios:

- Lectura NFC exitosa: envía la línea: UID CONCATENADO:XXXXXXXX (UID en hexadecimal concatenado, sin separadores, en mayúsculas).
- Botón presionado: envía la línea. BOTON:1

Funciones principales del firmware

Función	Descripción
setup()	Inicializa serial (115200 baud), configura GPIO25 como INPUT_PULLUP e inicia el PN532. Detiene la ejecución si el módulo no responde.
leerNFC()	Llama a readPassiveTargetID() con timeout de 100 ms. Construye el UID como string HEX concatenado, aplica cooldown de 2500 ms para el mismo UID y emite "UID CONCATENADO:XXXX" por Serial.
leerBoton()	Lee el estado de GPIO25. Si detecta LOW (botón presionado) y han pasado más de 300 ms desde la última pulsación, emite "BOTON:1" por Serial.

Código completo (Arduino IDE)

```
#include <Wire.h>
#include <Adafruit_PN532.h>

// Pines que se usaron entre el boton y el nfc
#define PIN_BOTON 25
#define PN532_SDA 21
#define PN532_SCL 22

// PN532 por I2C
Adafruit_PN532 nfc(PN532_SDA, PN532_SCL);

// Antirrebote boton
unsigned long ultimoPresionado = 0;
const unsigned long DEBOUNCE_MS = 300;

// Antilectura repetida nfc
String ultimoUID = "";
```

```

unsigned long tsUltimoUID = 0;
const unsigned long COOLDOWN_NFC_MS = 2500; // ms entre lecturas del mismo UID

void setup() {
    Serial.begin(115200);
    delay(200);

    // Botón con pull-up interno → LOW cuando está presionado
    pinMode(PIN_BOTON, INPUT_PULLUP);

    // Iniciar PN532
    nfc.begin();
    uint32_t versiondata = nfc.getFirmwareVersion();
    if (!versiondata) {
        Serial.println("ERROR:PN532 no encontrado");
        while (1) delay(500); // Detener si no hay lector
    }

    nfc.SAMConfig(); // Modo normal
    Serial.println("LISTO");
}

void loop() {
    leerNFC();
    leerBoton();
}

void leerNFC() {
    uint8_t uid[7];
    uint8_t uidLen = 0;

    // Espera no bloqueante: timeout 100ms para no trabar el loop
    bool encontrado = nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &uidLen,
100);

    if (!encontrado || uidLen == 0) return;

    // Construir UID como string hexadecimal concatenado (sin espacios, mayúsculas)
    String uidStr = "";
    for (uint8_t i = 0; i < uidLen; i++) {
        if (uid[i] < 0x10) uidStr += "0";
        uidStr += String(uid[i], HEX);
    }
}

```

```

uidStr.toUpperCase();

// Cooldown: ignorar si es el mismo UID dentro de los últimos COOLDOWN_NFC_MS
unsigned long ahora = millis();
if (uidStr == ultimoUID && (ahora - tsUltimoUID) < COOLDOWN_NFC_MS) return;

ultimoUID = uidStr;
tsUltimoUID = ahora;

Serial.print("UID CONCATENADO:");
Serial.println(uidStr);
}

void leerBoton() {
  if (digitalRead(PIN_BOTON) == LOW) {
    unsigned long ahora = millis();
    if ((ahora - ultimoPresionado) > DEBOUNCE_MS) {
      ultimoPresionado = ahora;
      Serial.println("BOTON:1");
    }
  }
}

```

7. Código Python — Aplicación de Escritorio (app_reciclaje_nfc_v2.py)

La aplicación Python es una interfaz gráfica de escritorio desarrollada con Tkinter. Gestiona toda la lógica del sistema: login por NFC, registro de usuarios nuevos, otorgamiento de puntos y visualización del historial de reciclaje.

Dependencias

```
pip install pyserial mysql-connector-python
```

Arquitectura de pantallas

- PantallaEspera: estado inicial. Muestra "Acerca tu tarjeta NFC". Escucha el puerto serial del ESP32 en un hilo paralelo (daemon thread). Gestiona la conexión a MySQL y asegura que la tabla usuarios_nfc exista.
- DialogoRegistrarNFC: ventana modal emergente que aparece cuando se detecta un UID desconocido. El usuario captura nombre, apellido, email y teléfono. Al guardar, crea el registro en usuarios y lo vincula en usuarios_nfc, luego inicia sesión automáticamente.

- **PantallaPrincipal:** pantalla activa tras el login. Muestra los puntos acumulados del usuario, un historial de reciclaje reciente y el panel del botón físico. Escucha "BOTON:1" del ESP32 para registrar una botella reciclada.

Flujo completo del sistema

#	Paso del flujo
1	La pantalla de espera se muestra al iniciar la app. El usuario selecciona el puerto COM y pulsa Conectar.
2	El ESP32 envía "UID CONCATENADO:XXXXXXXX" al acercar una tarjeta. Python recibe la línea en el hilo serial.
3.1	Si el UID ya existe en usuarios_nfc - login directo. Se abre PantallaPrincipal con los datos del usuario.
3.2	Si el UID es desconocido - se abre DialogoRegistrarNFC. El usuario llena sus datos, se crea el registro y se vincula la tarjeta, luego se hace login automático.
4	Con la sesión iniciada, el usuario deposita una botella. El ESP32 envía "BOTON:1". Python detecta el mensaje, inserta un registro en registro_reciclaje y suma los puntos al usuario.
5	La pantalla muestra el nuevo total de puntos con un flash verde de confirmación. El historial se actualiza en tiempo real.
6	El usuario pulsa "Cerrar sesión" y la app vuelve a PantallaEspera, lista para el siguiente usuario.

Aspectos clave del código Python

El código completo estará aparte ya que son muchas líneas del código, pero aquí se explicará lo más importante que se uso

- **Concurrencia:** Hilo serial daemon: la lectura del puerto COM corre en un thread paralelo para no bloquear la interfaz gráfica. Usa `root.after(0, lambda)` para actualizar la UI de forma thread-safe.
- **Anti-lectura repetida:** Cooldown por UID: la pantalla de espera ignora el mismo UID durante 3 segundos para evitar que una tarjeta apoyada sobre el lector genere múltiples intentos de login.
- **Autocreación de tabla:** `asegurar_tabla_nfc()`: al arrancar, ejecuta `CREATE TABLE IF NOT EXISTS` para garantizar que la tabla `usuarios_nfc` existe sin importar el estado previo de la base de datos.
- **Consistencia de datos:** Transacción atómica de puntos: al registrar una botella, se ejecutan `INSERT` en `registro_reciclaje` y `UPDATE` en `usuarios` dentro de la misma conexión antes de hacer `commit`.

- **Escalabilidad:** Los puntos otorgados se toman del campo puntos de la tabla tipos_botella (id_tipo=1 con 1 botón). La arquitectura permite 2 botones en el futuro con mínimos cambios.

8. Base de Datos — nfc_reciclaje (MySQL)

La base de datos almacena usuarios, tarjetas NFC, tipos de botella y el historial de reciclaje. Se importa en Laragon ejecutando el archivo nfc_reciclaje_v2.sql desde HeidiSQL o phpMyAdmin.

Tablas del sistema

Tabla	Columnas clave	Descripción
usuarios	id_usuario, nombre, apellido, email, puntos_totales, estado	Participantes del sistema. La columna pin existe por compatibilidad pero ya no se utiliza para el login (el acceso es 100% por tarjeta NFC).
usuarios_nfc	id, id_usuario (FK), uid_nfc (UNIQUE)	Vincula cada tarjeta NFC a un usuario. Un usuario puede tener varias tarjetas. El UID se almacena como hexadecimal concatenado en mayúsculas (ej. 0D1B1207).
tipos_botella	id_tipo, nombre, capacidad_ml, puntos	Catálogo de tipos de botella. Con 1 botón la app usa siempre id_tipo=1 (600ml, 5 puntos). Con 2 botones se usa también id_tipo=2 (1L, 10 puntos).
registro_reciclaje	id_registro, id_usuario, id_tipo, uid_leido, puntos_otorgados, fecha_hora	Historial de eventos. uid_leido toma el valor 'BOTON' cuando el evento proviene del botón físico (no de una tarjeta de botella).

Vistas y procedimientos almacenados

- **Vista:** vista_ranking: ranking general de usuarios activos ordenado por puntos_totales DESC. Incluye conteo de botellas
- **Vista:** vista_historial_reciente: últimas 100 transacciones con nombre de usuario, tipo de botella y puntos otorgados, ordenadas por fecha_hora DESC.
- **Stored Procedure:** registrar_reciclaje(p_id_usuario, p_id_tipo): inserta en registro_reciclaje y actualiza puntos_totales de forma atómica. Devuelve puntos_otorgados y puntos_totales_nuevos.

- **Stored Procedure:** vincular_tarjeta(p_id_usuario, p_uid_nfc): asocia un UID NFC a un usuario existente usando INSERT IGNORE para evitar duplicados.

Obtener el UID de una tarjeta nueva

Para conocer el UID de una tarjeta que aún no está registrada: abrir el Monitor Serial del Arduino IDE a 115200 baudios y acercar la tarjeta al PN532. Aparecerá una línea como:

```
UID CONCATENADO:0D1B1207
```

Ese valor (sin los dos puntos del prefijo) es el UID que se almacena en usuarios_nfc.uid_nfc. La aplicación Python lo registra automáticamente al usar el formulario de registro.

9. Conclusiones

Eduardo: El desarrollo de EcoPoints permitió integrar de manera práctica la tecnología NFC con una base de datos relacional y una interfaz gráfica de escritorio. La arquitectura de login por tarjeta NFC demuestra cómo un simple identificador de 4 bytes puede reemplazar contraseñas y formularios complejos, haciendo el sistema más rápido y accesible. La experiencia confirmó que la ESP32 es una plataforma muy versátil capaz de manejar simultáneamente lectura NFC y detección de botón sin comprometer la fluidez de la comunicación serial.

Itzel: Esta práctica mostró cómo una tecnología cotidiana como NFC puede ser la base de un sistema completo de gamificación ambiental. El manejo del registro automático de tarjetas desconocidas fue particularmente enriquecedor: en lugar de rechazar la tarjeta, el sistema invita al usuario a registrarse, lo que mejora la experiencia de usuario y facilita la adopción del sistema. Integrar la interfaz gráfica con el puerto serial en tiempo real también reforzó conceptos de programación concurrente con hilos.

Damian: El proyecto EcoPoints demostró la viabilidad de sistemas de reciclaje con incentivos digitales. El botón físico de dos pines con INPUT_PULLUP resultó una solución robusta y económica para registrar eventos de reciclaje sin sensores adicionales. La separación del firmware (ESP32) y la lógica de negocio (Python + MySQL) facilita enormemente el mantenimiento y la escalabilidad del sistema: agregar un segundo botón o nuevos tipos de botella requiere cambios mínimos en el código, lo que valida el diseño orientado a la extensibilidad.

10. Videos, GitHub y Evidencias

Evidencias del proyecto: capturas de pantalla de la aplicación en funcionamiento, el Monitor Serial con la lectura de tarjetas y el registro de puntos en la base de datos en un video y el enlace del GitHub.

GitHub del proyecto:

https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/NFC_reciclaje_proyecto

Video de demostración: <https://youtu.be/eAWvPgXPVLo>